The Mathematics of Gradient Boosting Regression

Demystifying the algorithm step-by-step

1. Initialize the Model

Start with a base model, often the mean of the target values:

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^{n} L(y_i, \gamma)$$

Annotated: The slate blue curve represents this initial base prediction.

2. Compute Residuals

For each subsequent iteration m from 1 to M, compute the negative gradient of the loss function (pseudo-residuals):

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$$

Annotated: The difference between the current

3. Fit a Weak Learner

Fit a simple model (e.g., a regression tree) $h_m(x)$ to the current residuals r_{im}:

$$h_m(x) \approx \{r_{im}\}_{i=1}^{n}$$

3. Fit a Weak Learner

Fit a simple model (e.g., a regression tree) $h_m(x)$ to the current residuals r_{im}:

$$h_m(x) \approx \{r_{im}\}_{i=1}^{n}$$

Annotated: The sage green curve; trains the errors.

4. Update the Model

Find the optimal learning rate γ_m and update the additive model:

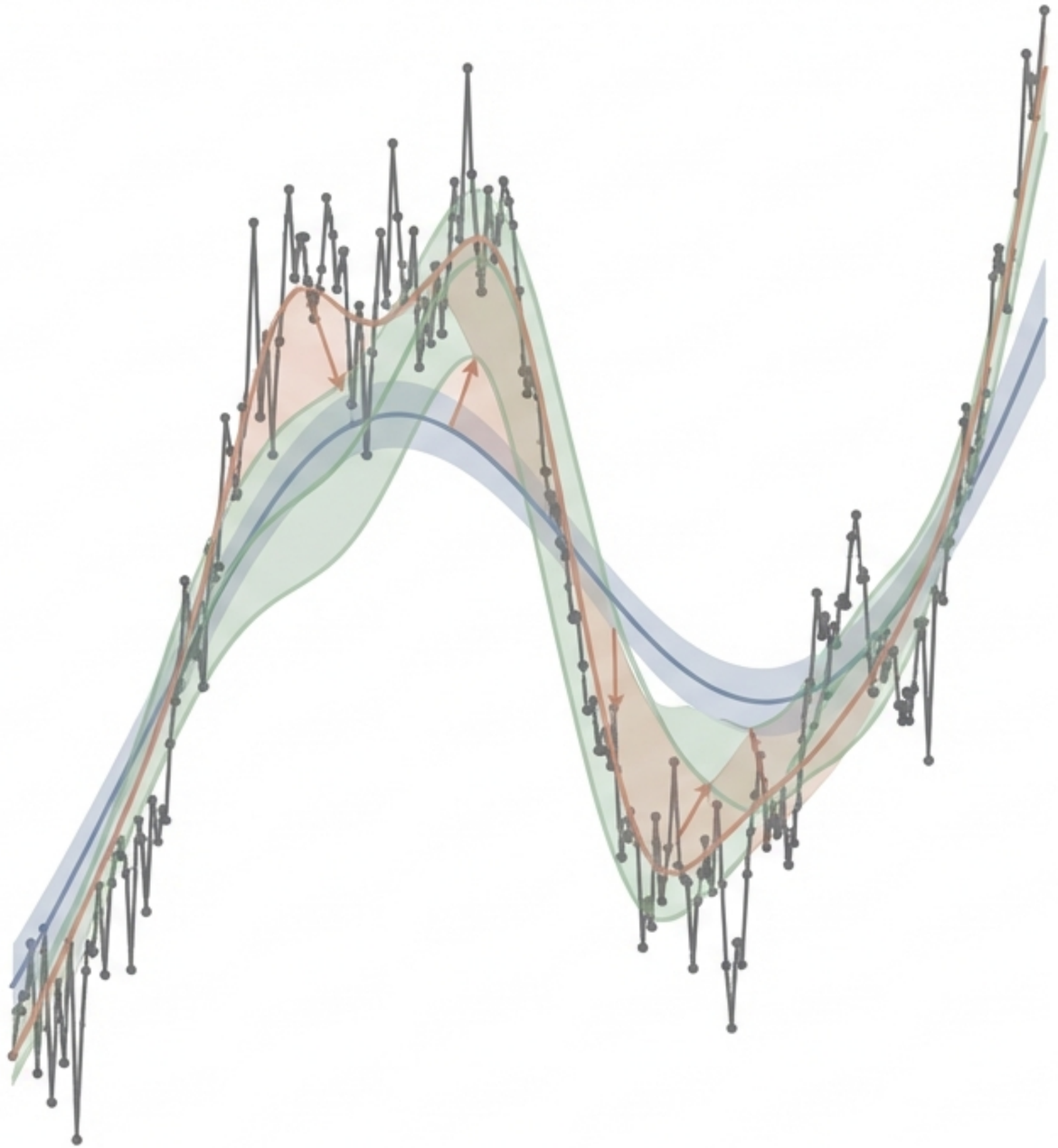
$$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^{n} L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$$

$$F_m(x) = F_{m-1}(x) + \nu\gamma_m h_m(z)$$

5. Iterate until Convergence

Repeat steps 2-4, gradually reducing the errors. The final model is a weighted sum:

$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \nu\gamma_m h_m(x)$$

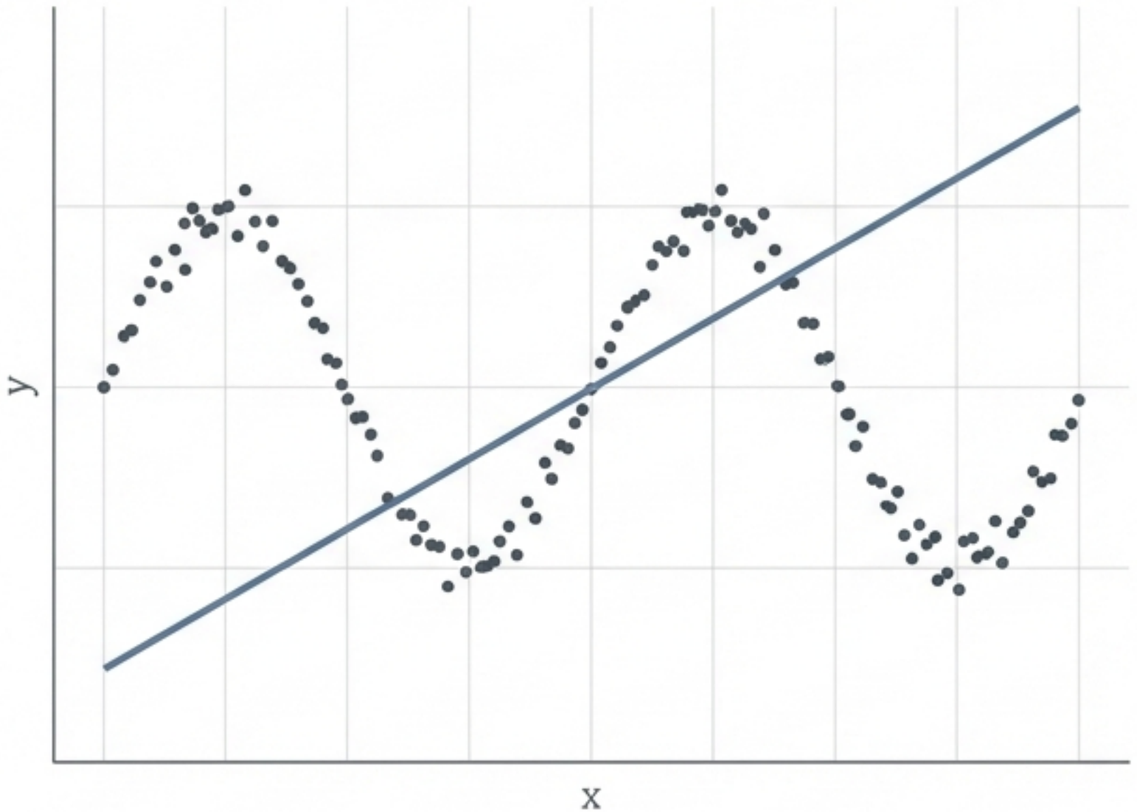


A purely visual document designed for reading.

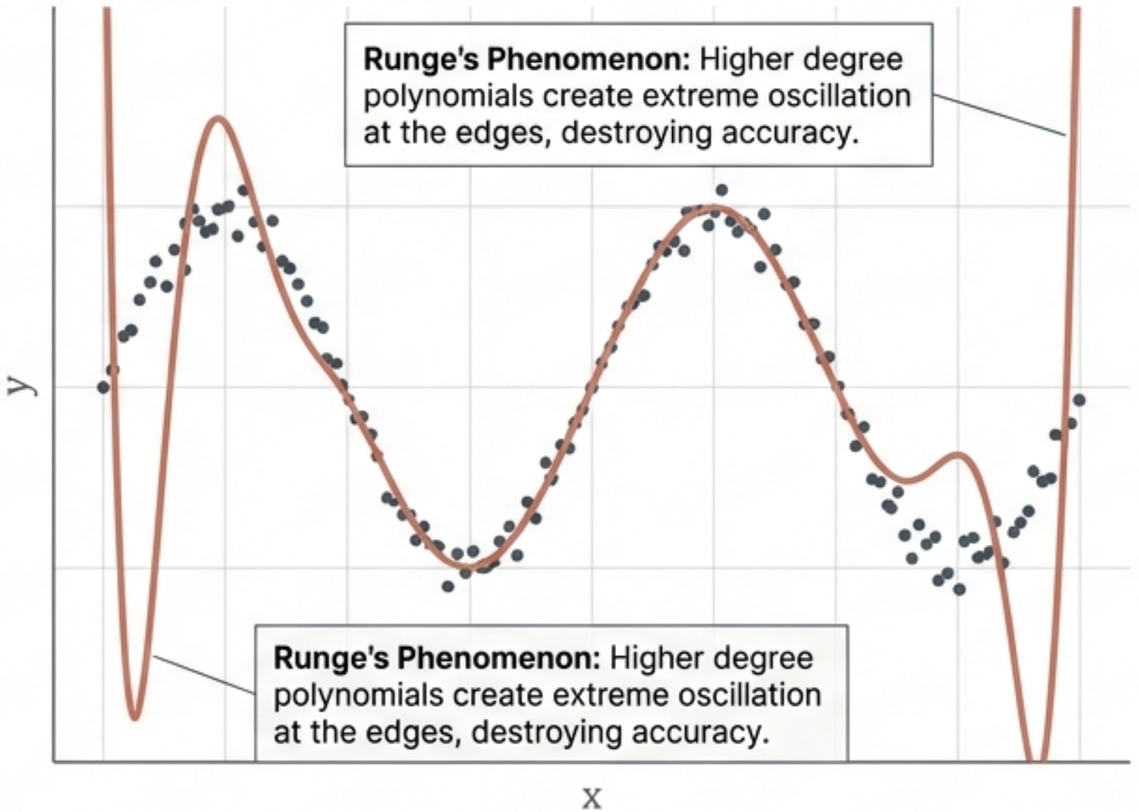
Why Traditional Models Fall Short

When data is highly non-linear, single, rigid formulas cannot capture the true relationship.

Linear Regression: Too Simple

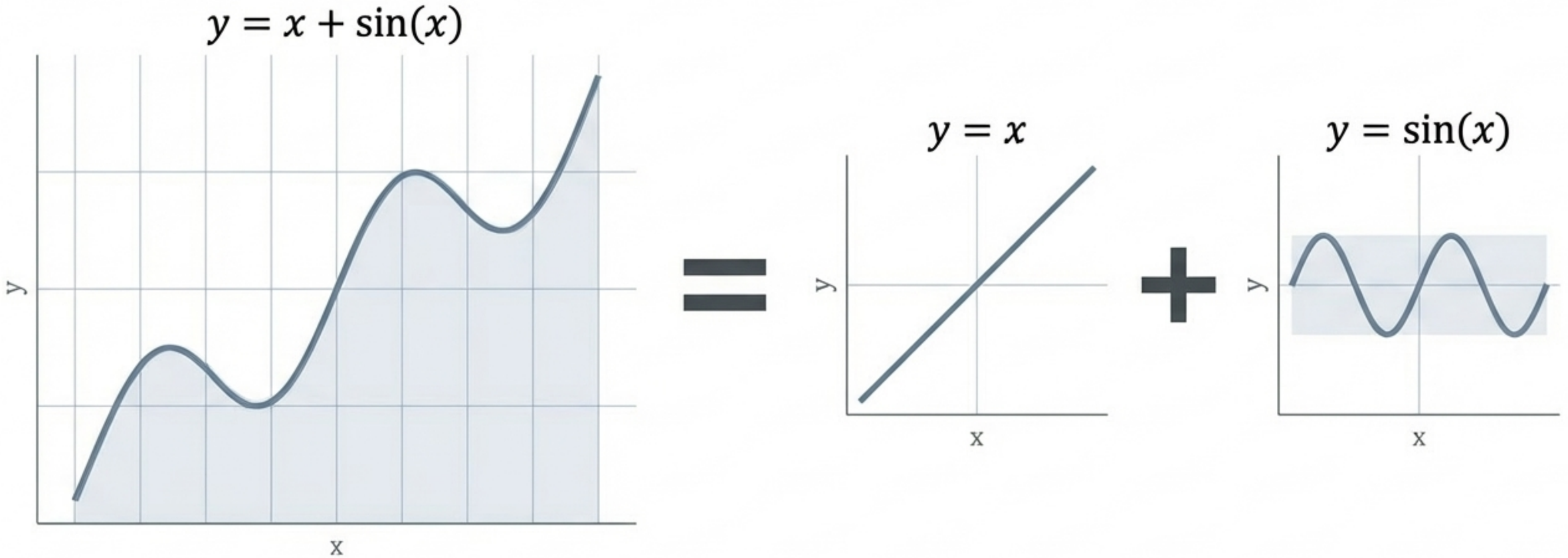


Polynomial Regression: Runge's Phenomenon



The Master Key: Additive Modelling

Instead of searching for one massive, complex equation, Additive Modelling breaks the problem down. We combine multiple, smaller, simpler functions to approximate the complex reality.



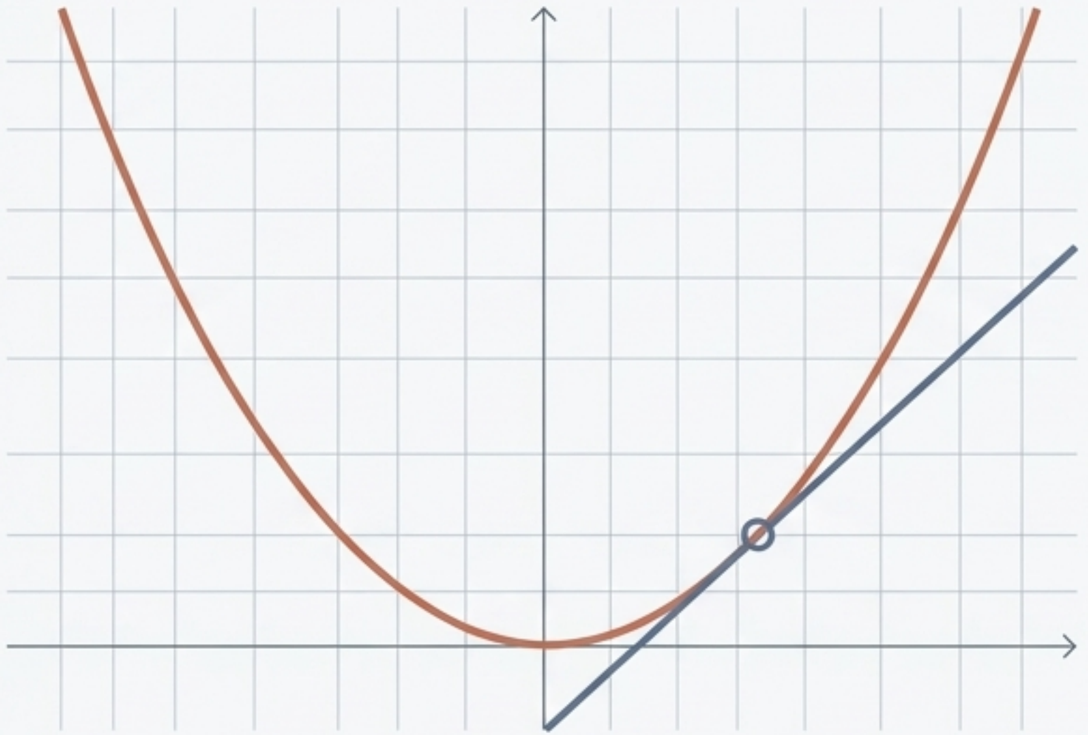
The Two Prerequisites

To begin, we need historical data to train on, and a way to mathematically measure our errors (the loss function) at every single point.

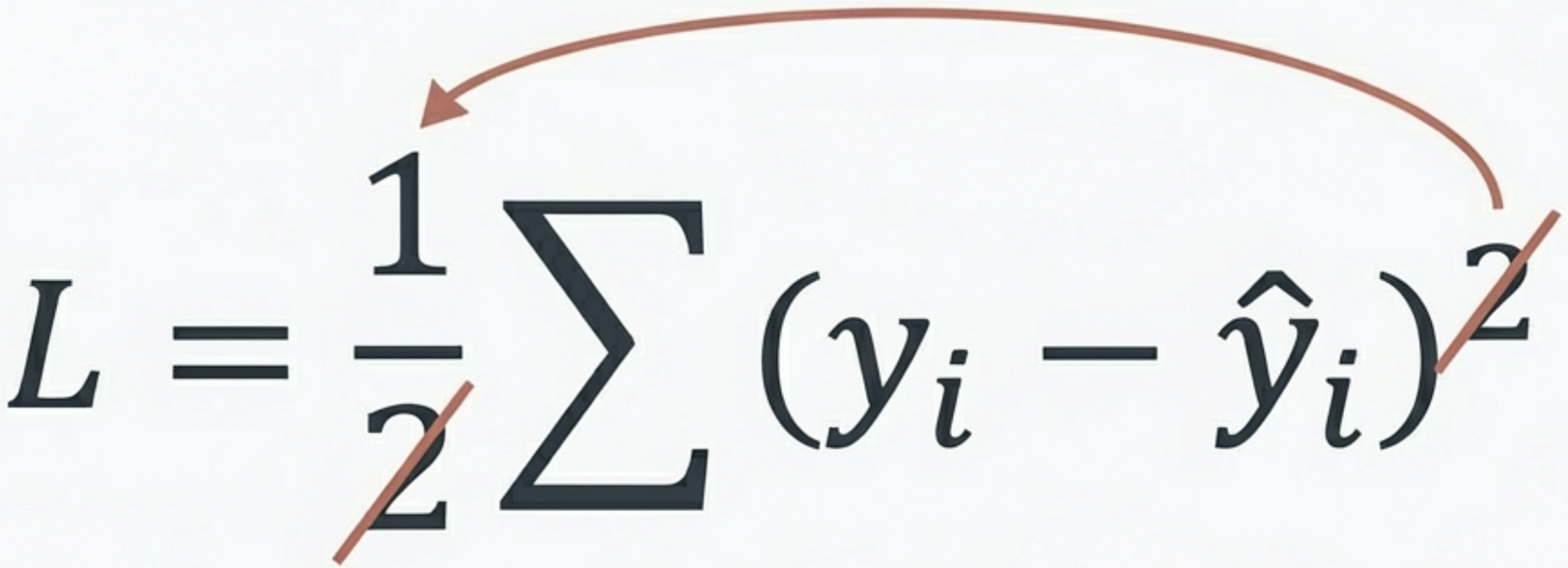
1. Training Dataset

Startup	R&D	Admin	Marketing	Profit
Startup 1	• • •	•	•	192
Startup 2	• • •	•	•	144
Startup 3	• • •	•	•	92

2. A Differentiable Loss Function.



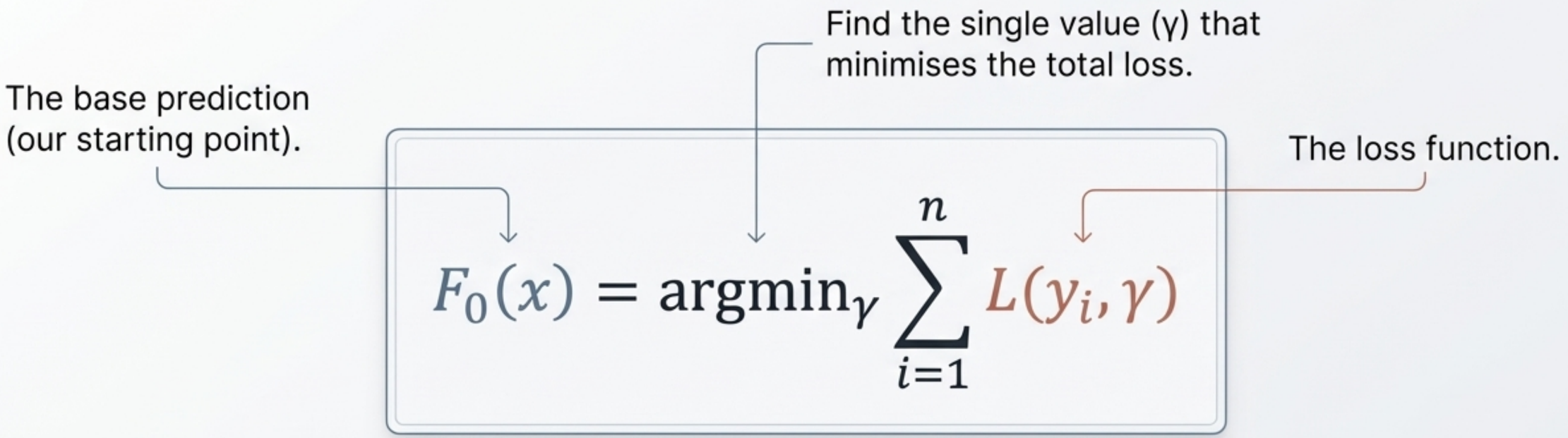
The Loss Function: Half Mean Squared Error



Why the 1/2 multiplier? Pure mathematical convenience. When we apply calculus later, the chain rule brings the exponent 2 down, cleanly cancelling out the 1/2. It simplifies the maths without changing which predictions are fundamentally better or worse.

Step 1: Initialising the Model (The Formal Maths)

The algorithm begins by finding a single, flat constant value that produces the lowest possible error across all data points before any trees are built.



Step 1: Translated to Plain English

Despite the complex calculus, Step 1 is simply asking for the average of your target column. Our starting prediction for all startups is 142.66.

The Translation

$$\frac{d}{d\gamma}\left[\frac{1}{2}\sum(y_i-\gamma)^2\right]=0$$

$$\downarrow$$

$$\sum(y_i-\gamma)*(-1)=0$$

$$\downarrow$$

$$\boldsymbol{\gamma = \text{Mean of } y}$$

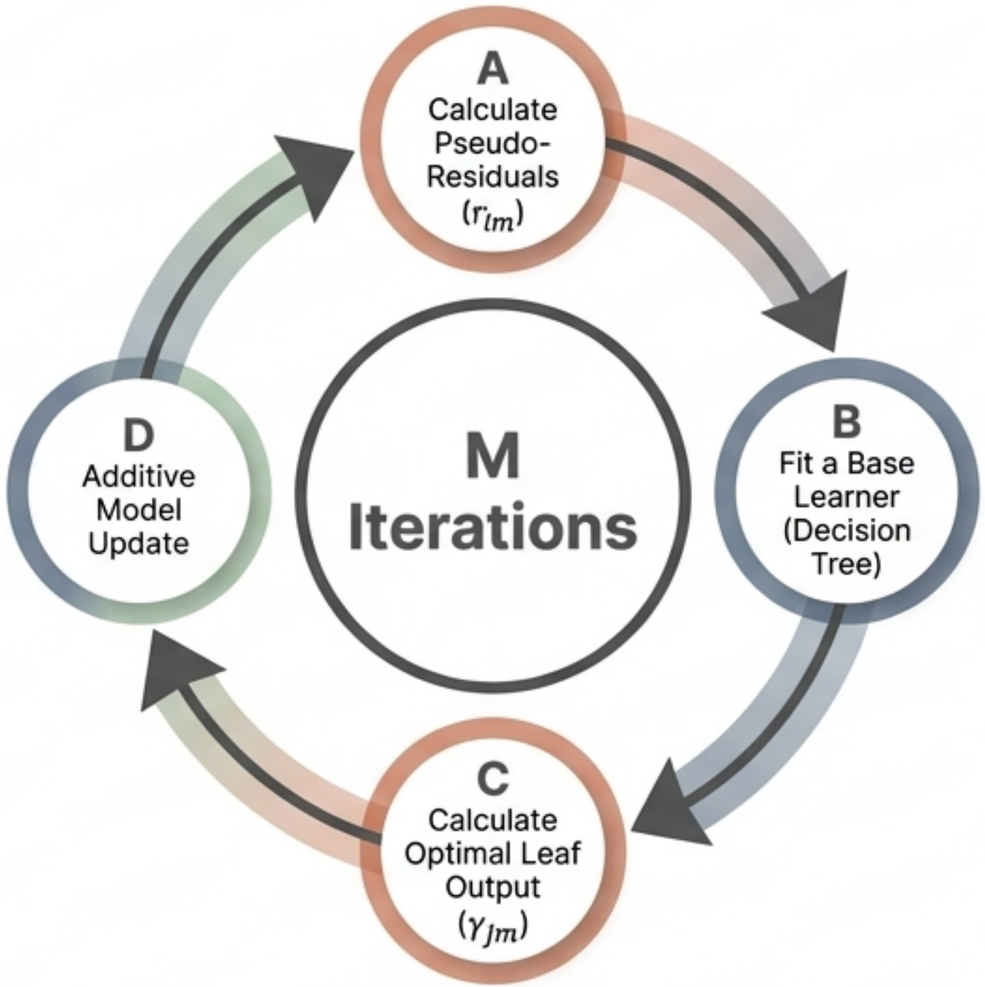
The Application

Startup	Profit
Startup 1:	192
Startup 2:	144
Startup 3:	92

$$\frac{192+144+92}{3}=142.66$$

Step 2: The Boosting Loop

Having established our base prediction, we enter a sequential loop. We build M decision trees, one after another, exclusively focused on fixing the errors of the trees that came before it.



Step 2(a): Calculating Pseudo-Residuals (The Formal (The Formal Maths)

We calculate how far off our current predictions are by finding the negative derivative of the loss function. This tells the algorithm what direction to step in to reduce the error.

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}$$

The pseudo-residual for a specific row and specific tree.

The negative gradient (derivative) of the loss function.

Step 2(a): Translated to Plain English

For a squared-error loss function, the negative gradient simplifies wonderfully to just Actual Output minus Predicted Output.

The Translation

$$\frac{d}{dF}\left[\frac{1}{2}(y-F)^2\right]$$

↓

$$-1 * (y-F) * -1$$

↓

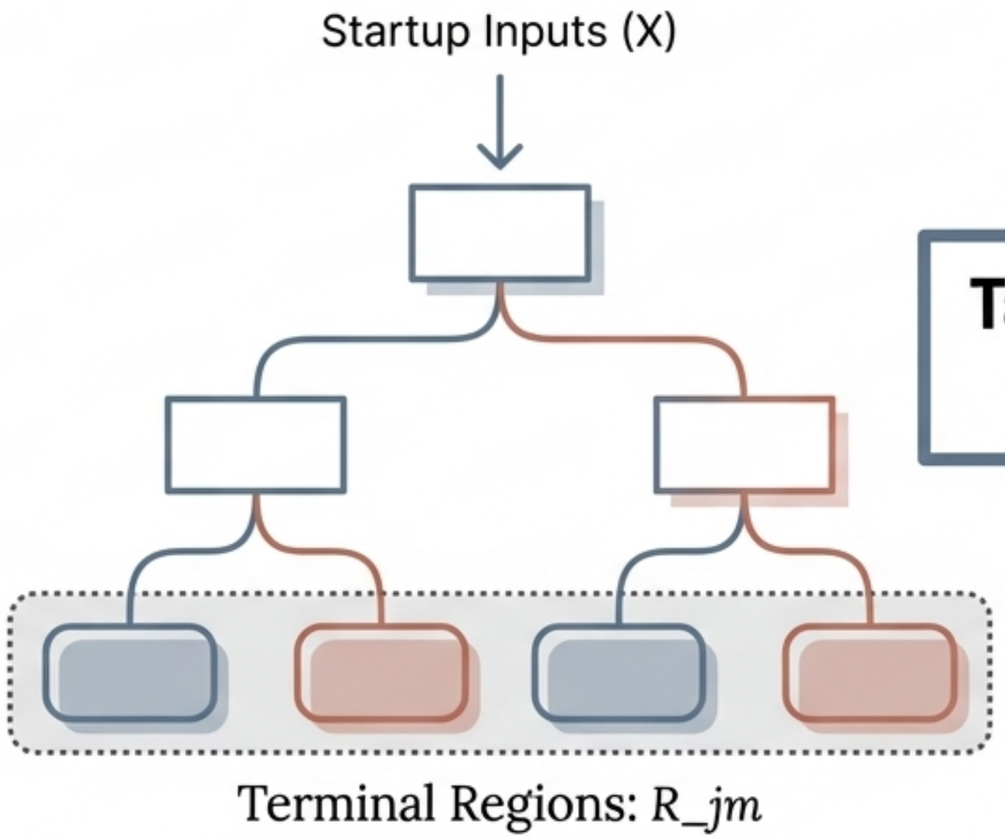
$$\boldsymbol{r_{im} = y_i - \hat{y}_i}$$

The Application

Startup	Actual (y)	Predicted ($\hat{y}$)	Residual (r)
1	192	− 142.66 =	49.34
2	144	− 142.66 =	1.34
3	92	− 142.66 =	-50.66

Step 2(b): Fitting the Base Learner

We train a weak Decision Tree. Crucially, this tree does not predict the startup's profit. It predicts the **residual**—the gap between our current model and reality. The tree groups the startups into specific terminal regions (leaves).



Target: Pseudo-Residuals (r_im), NOT Profit

Step 2(c): Calculating Leaf Outputs (The Formal Maths)

For every single leaf in the newly created tree, we must calculate the precise numerical output that will minimise the loss for just the data points inside that leaf.

$$\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

The optimal output value for a specific leaf.

Only looking at the data points that fell into this specific leaf.

Previous prediction plus the new leaf output.

Step 2(c): Translated to Plain English

Because we are using a squared-error loss function, the complex calculus simplifies again. The optimal output is simply the average of the residuals that fall into that leaf.

The Translation

$$\frac{d}{d\gamma}[Loss] = 0$$

$$\downarrow$$

γ_{jm} **= Mean of the residuals in the leaf**

The Application

49.34
1.34

$$\rightarrow \quad \frac{49.34 + 1.34}{2} = 25.34$$

Step 2(d): Updating the Model

The additive update. We take our previous model, and we add the predictions from the newly minted decision tree. With this step complete, the model's **overall error has shrunk**, and the loop begins again for the next tree.

$$F_m(x) = F_{m-1}(x) + \gamma_{jm}$$

New Model = Old Model + New Tree Predictions

The Final Architecture

By defining a clear loss function and letting calculus guide our steps, we successfully build a highly accurate composite model. What started as intimidating notation is fundamentally just a sequence of calculating means, finding residuals, and taking small, additive steps toward reality.

$$F_M(x) = F_0(x) + Tree_1(x) + Tree_2(x) + \cdots + Tree_M(x)$$

$F_0(x)$ $F_0(x) + Tree_1(x)$ $F_M(x)$